

lab5

March 30, 2026

1 Głębokie Uczenie w Praktyce

1.1 Laboratorium 4

1.1.1 Mateusz Horczak

```
[15]: import os
import matplotlib.pyplot as plt
from tensorflow.keras import models, layers, optimizers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image
```

```
[16]: train_dir = 'train-test-valid/train'
validation_dir = 'train-test-valid/validation'
test_dir = 'train-test-valid/test'

train_cats_dir = os.path.join(train_dir, 'cats')
train_dogs_dir = os.path.join(train_dir, 'dogs')
validation_cats_dir = os.path.join(validation_dir, 'cats')
validation_dogs_dir = os.path.join(validation_dir, 'dogs')
test_cats_dir = os.path.join(test_dir, 'cats')
test_dogs_dir = os.path.join(test_dir, 'dogs')
```

```
[20]: print("Liczba obrazów treningowych kotów:", len(os.listdir(train_cats_dir)))
print("Liczba obrazów treningowych psów:", len(os.listdir(train_dogs_dir)))
print("Liczba obrazów walidacyjnych kotów:", len(os.
↪listdir(validation_cats_dir)))
print("Liczba obrazów walidacyjnych psów:", len(os.
↪listdir(validation_dogs_dir)))
print("Liczba obrazów testowych kotów:", len(os.listdir(test_cats_dir)))
print("Liczba obrazów testowych psów:", len(os.listdir(test_dogs_dir)))
```

```
Liczba obrazów treningowych kotów: 1000
Liczba obrazów treningowych psów: 1000
Liczba obrazów walidacyjnych kotów: 500
Liczba obrazów walidacyjnych psów: 500
Liczba obrazów testowych kotów: 500
Liczba obrazów testowych psów: 500
```

Ćwiczenie 1 Tworzenie ImageDataGenerator

```
[18]: datagen = ImageDataGenerator(
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode='nearest'
    )

    fnames = [os.path.join(train_cats_dir, fname) for fname in os.
        ↳listdir(train_cats_dir)]
    img_path = fnames[0]

    img = image.load_img(img_path, target_size=(150, 150))
    x = image.img_to_array(img)
    x = x.reshape((1,) + x.shape)

    plt.figure(figsize=(12, 8))
    for i, batch in enumerate(datagen.flow(x, batch_size=1), 1):
        plt.subplot(2, 3, i)
        plt.imshow(image.array_to_img(batch[0]))
        plt.axis('off')
        if i >= 6:
            break
    plt.suptitle('Ćwiczenie 1 - Augmentowane wersje tego samego obrazu')
    plt.show()
```

Ćwiczenie 1 - Augmentowane wersje tego samego obrazu



Ćwiczenie 2 Użycie ImageDataGenerator do generacji danych treningowych i walidacyjnych

```
[19]: # z poprzedniego labu
model_aug = models.Sequential()
model_aug.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)))
model_aug.add(layers.MaxPooling2D((2, 2)))
model_aug.add(layers.Conv2D(64, (3, 3), activation='relu'))
model_aug.add(layers.MaxPooling2D((2, 2)))
model_aug.add(layers.Conv2D(128, (3, 3), activation='relu'))
model_aug.add(layers.MaxPooling2D((2, 2)))
model_aug.add(layers.Conv2D(128, (3, 3), activation='relu'))
model_aug.add(layers.MaxPooling2D((2, 2)))
model_aug.add(layers.Flatten())
model_aug.add(layers.Dense(512, activation='relu'))
model_aug.add(layers.Dense(1, activation='sigmoid'))

model_aug.compile(loss='binary_crossentropy',
                  optimizer=optimizers.RMSprop(learning_rate=1e-4),
                  metrics=['acc'])
```

```
model_aug.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_16 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_16 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_17 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_17 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_18 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_18 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_19 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_19 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten_4 (Flatten)	(None, 6272)	0
dense_8 (Dense)	(None, 512)	3,211,776
dense_9 (Dense)	(None, 1)	513

Total params: 3,453,121 (13.17 MB)

Trainable params: 3,453,121 (13.17 MB)

Non-trainable params: 0 (0.00 B)

```
[7]: train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=40,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True
```

```

)
test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir, target_size=(150, 150), batch_size=32, class_mode='binary')
validation_generator = test_datagen.flow_from_directory(
    validation_dir, target_size=(150, 150), batch_size=32, class_mode='binary')

history_aug = model_aug.fit(
    train_generator,
    steps_per_epoch=100,
    epochs=30,
    validation_data=validation_generator,
    validation_steps=50
)

```

Found 2000 images belonging to 2 classes.

Found 1000 images belonging to 2 classes.

Epoch 1/30

63/100 1:07 2s/step -
acc: 0.5228 - loss: 0.6952

/home/mateusz/PycharmProjects/glebokie_uczenie_w_praktyce/.venv/lib/python3.12/site-packages/keras/src/trainers/epoch_iterator.py:116: UserWarning: Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate at least `steps\_per\_epoch \* epochs` batches. You may need to use the `repeat()` function when building your dataset.

self._interrupted_warning()

100/100 182s 2s/step -
acc: 0.5350 - loss: 0.6915 - val_acc: 0.5000 - val_loss: 0.6939

Epoch 2/30

100/100 63s 624ms/step -
acc: 0.5600 - loss: 0.6825 - val_acc: 0.5830 - val_loss: 0.6694

Epoch 3/30

100/100 54s 536ms/step -
acc: 0.5505 - loss: 0.6773 - val_acc: 0.5650 - val_loss: 0.6764

Epoch 4/30

100/100 54s 536ms/step -
acc: 0.5770 - loss: 0.6692 - val_acc: 0.5370 - val_loss: 0.6858

Epoch 5/30

100/100 48s 473ms/step -
acc: 0.5890 - loss: 0.6627 - val_acc: 0.6130 - val_loss: 0.6441

Epoch 6/30

100/100 79s 446ms/step -
acc: 0.5995 - loss: 0.6584 - val_acc: 0.5290 - val_loss: 0.6761

Epoch 7/30

100/100 48s 478ms/step -
acc: 0.6230 - loss: 0.6445 - val_acc: 0.6030 - val_loss: 0.6593

Epoch 8/30
100/100 48s 478ms/step -
acc: 0.6385 - loss: 0.6250 - val_acc: 0.6680 - val_loss: 0.5995
Epoch 9/30
100/100 49s 486ms/step -
acc: 0.6655 - loss: 0.6168 - val_acc: 0.6440 - val_loss: 0.6149
Epoch 10/30
100/100 83s 501ms/step -
acc: 0.6530 - loss: 0.6203 - val_acc: 0.6580 - val_loss: 0.6005
Epoch 11/30
100/100 76s 440ms/step -
acc: 0.6610 - loss: 0.6005 - val_acc: 0.6690 - val_loss: 0.5940
Epoch 12/30
100/100 53s 525ms/step -
acc: 0.6645 - loss: 0.6023 - val_acc: 0.6500 - val_loss: 0.6274
Epoch 13/30
100/100 47s 465ms/step -
acc: 0.6785 - loss: 0.6024 - val_acc: 0.6560 - val_loss: 0.6127
Epoch 14/30
100/100 79s 441ms/step -
acc: 0.6805 - loss: 0.5937 - val_acc: 0.6870 - val_loss: 0.5825
Epoch 15/30
100/100 48s 472ms/step -
acc: 0.6815 - loss: 0.5914 - val_acc: 0.6930 - val_loss: 0.5657
Epoch 16/30
100/100 79s 440ms/step -
acc: 0.6830 - loss: 0.5833 - val_acc: 0.6820 - val_loss: 0.5782
Epoch 17/30
100/100 52s 522ms/step -
acc: 0.6910 - loss: 0.5826 - val_acc: 0.6760 - val_loss: 0.5662
Epoch 18/30
100/100 46s 460ms/step -
acc: 0.6990 - loss: 0.5803 - val_acc: 0.6990 - val_loss: 0.5556
Epoch 19/30
100/100 47s 468ms/step -
acc: 0.7020 - loss: 0.5717 - val_acc: 0.7030 - val_loss: 0.5495
Epoch 20/30
100/100 84s 490ms/step -
acc: 0.7075 - loss: 0.5676 - val_acc: 0.7080 - val_loss: 0.5485
Epoch 21/30
100/100 47s 463ms/step -
acc: 0.7015 - loss: 0.5661 - val_acc: 0.7210 - val_loss: 0.5466
Epoch 22/30
100/100 47s 464ms/step -
acc: 0.7125 - loss: 0.5505 - val_acc: 0.7200 - val_loss: 0.5373
Epoch 23/30
100/100 47s 469ms/step -
acc: 0.7145 - loss: 0.5513 - val_acc: 0.7180 - val_loss: 0.5434

```

Epoch 24/30
100/100          47s 466ms/step -
acc: 0.7075 - loss: 0.5620 - val_acc: 0.7100 - val_loss: 0.5415
Epoch 25/30
100/100          57s 563ms/step -
acc: 0.7165 - loss: 0.5516 - val_acc: 0.7120 - val_loss: 0.5295
Epoch 26/30
100/100          47s 465ms/step -
acc: 0.7210 - loss: 0.5422 - val_acc: 0.6950 - val_loss: 0.5722
Epoch 27/30
100/100          47s 468ms/step -
acc: 0.7085 - loss: 0.5516 - val_acc: 0.6980 - val_loss: 0.5563
Epoch 28/30
100/100          47s 461ms/step -
acc: 0.7195 - loss: 0.5481 - val_acc: 0.7030 - val_loss: 0.5539
Epoch 29/30
100/100          48s 471ms/step -
acc: 0.7270 - loss: 0.5413 - val_acc: 0.7190 - val_loss: 0.5445
Epoch 30/30
100/100          47s 465ms/step -
acc: 0.7240 - loss: 0.5413 - val_acc: 0.7320 - val_loss: 0.5252

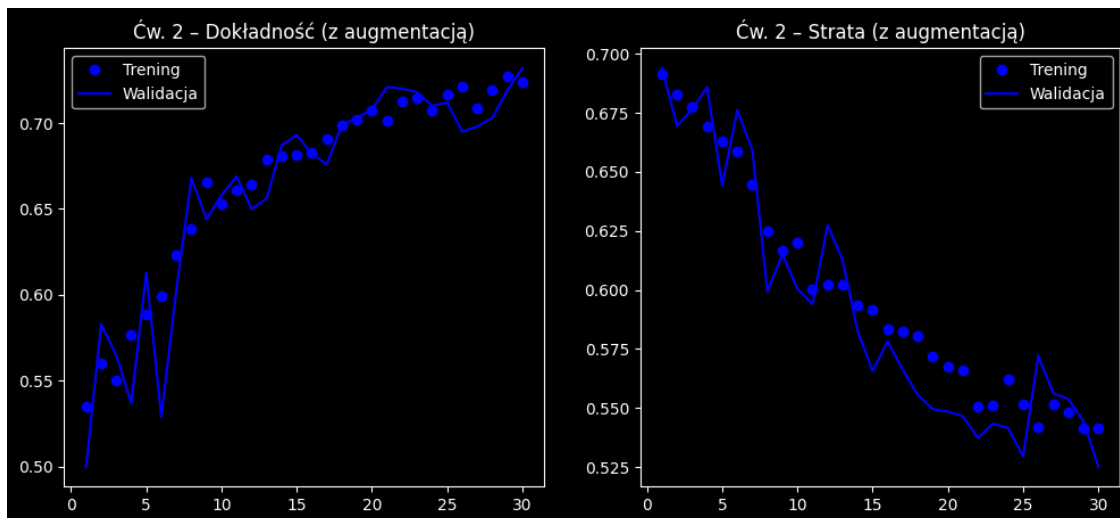
```

```

[8]: acc = history_aug.history['acc']
val_acc = history_aug.history['val_acc']
loss = history_aug.history['loss']
val_loss = history_aug.history['val_loss']
epochs = range(1, len(acc) + 1)

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs, acc, 'bo', label='Trening')
plt.plot(epochs, val_acc, 'b', label='Walidacja')
plt.title('Ćw. 2 - Dokładność (z augmentacją)')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(epochs, loss, 'bo', label='Trening')
plt.plot(epochs, val_loss, 'b', label='Walidacja')
plt.title('Ćw. 2 - Strata (z augmentacją)')
plt.legend()
plt.show()

```



Ćwiczenie 3 Wpływ warstwy Dropout na dopasowanie modelu (trenowanie bez augmentacji danych)

```
[9]: def create_model(dropout_rate=0.5):
    model = models.Sequential()
    model.add(layers.Input((150, 150, 3)))
    model.add(layers.Conv2D(32, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Conv2D(128, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Dropout(dropout_rate))
    model.add(layers.Conv2D(128, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Dropout(dropout_rate))
    model.add(layers.Flatten())
    model.add(layers.Dense(512, activation='relu'))
    model.add(layers.Dropout(dropout_rate))
    model.add(layers.Dense(1, activation='sigmoid'))
    return model
```

```
[10]: train_datagen_noaug = ImageDataGenerator(rescale=1./255)
train_generator_noaug = train_datagen_noaug.flow_from_directory(
    train_dir, target_size=(150, 150), batch_size=32, class_mode='binary')
```

Found 2000 images belonging to 2 classes.

Trening z Dropout = 0.2 (bez augmentacji)


```
[11]: model_02 = create_model(0.2)
model_02.compile(loss='binary_crossentropy',
                  optimizer=optimizers.RMSprop(learning_rate=1e-4),
                  metrics=['acc'])

history_02 = model_02.fit(
    train_generator_noaug,
    steps_per_epoch=100,
    epochs=30,
    validation_data=validation_generator,
    validation_steps=50
)
```

```
Epoch 1/30
100/100          45s 438ms/step -
acc: 0.5170 - loss: 0.6931 - val_acc: 0.6050 - val_loss: 0.6891
Epoch 2/30
100/100          43s 429ms/step -
acc: 0.5440 - loss: 0.6840 - val_acc: 0.5250 - val_loss: 0.6881
Epoch 3/30
100/100          80s 411ms/step -
acc: 0.5830 - loss: 0.6773 - val_acc: 0.5940 - val_loss: 0.6677
Epoch 4/30
100/100          43s 429ms/step -
acc: 0.5860 - loss: 0.6640 - val_acc: 0.5920 - val_loss: 0.6600
Epoch 5/30
100/100          44s 435ms/step -
acc: 0.6105 - loss: 0.6575 - val_acc: 0.6100 - val_loss: 0.6462
Epoch 6/30
100/100          79s 403ms/step -
acc: 0.6375 - loss: 0.6448 - val_acc: 0.5770 - val_loss: 0.6779
Epoch 7/30
100/100          43s 427ms/step -
acc: 0.6465 - loss: 0.6365 - val_acc: 0.6060 - val_loss: 0.6419
Epoch 8/30
100/100          44s 435ms/step -
acc: 0.6530 - loss: 0.6201 - val_acc: 0.6550 - val_loss: 0.6152
Epoch 9/30
100/100          79s 403ms/step -
acc: 0.6755 - loss: 0.6038 - val_acc: 0.5900 - val_loss: 0.6609
Epoch 10/30
100/100          44s 437ms/step -
acc: 0.6855 - loss: 0.5808 - val_acc: 0.6740 - val_loss: 0.5941
Epoch 11/30
100/100          43s 430ms/step -
acc: 0.7085 - loss: 0.5633 - val_acc: 0.6820 - val_loss: 0.5824
Epoch 12/30
100/100          44s 437ms/step -
```

acc: 0.7180 - loss: 0.5454 - val_acc: 0.6980 - val_loss: 0.5783
 Epoch 13/30
 100/100 79s 405ms/step -
 acc: 0.7350 - loss: 0.5232 - val_acc: 0.6880 - val_loss: 0.5806
 Epoch 14/30
 100/100 44s 434ms/step -
 acc: 0.7480 - loss: 0.5078 - val_acc: 0.6880 - val_loss: 0.5866
 Epoch 15/30
 100/100 43s 429ms/step -
 acc: 0.7465 - loss: 0.5005 - val_acc: 0.7000 - val_loss: 0.5755
 Epoch 16/30
 100/100 43s 431ms/step -
 acc: 0.7575 - loss: 0.4827 - val_acc: 0.7130 - val_loss: 0.5600
 Epoch 17/30
 100/100 85s 459ms/step -
 acc: 0.7780 - loss: 0.4726 - val_acc: 0.6930 - val_loss: 0.5949
 Epoch 18/30
 100/100 43s 424ms/step -
 acc: 0.7695 - loss: 0.4640 - val_acc: 0.7250 - val_loss: 0.5488
 Epoch 19/30
 100/100 85s 459ms/step -
 acc: 0.7890 - loss: 0.4519 - val_acc: 0.7210 - val_loss: 0.5423
 Epoch 20/30
 100/100 44s 441ms/step -
 acc: 0.7945 - loss: 0.4389 - val_acc: 0.7220 - val_loss: 0.5544
 Epoch 21/30
 100/100 79s 416ms/step -
 acc: 0.8130 - loss: 0.4200 - val_acc: 0.7120 - val_loss: 0.5674
 Epoch 22/30
 100/100 81s 403ms/step -
 acc: 0.8020 - loss: 0.4125 - val_acc: 0.6970 - val_loss: 0.6121
 Epoch 23/30
 100/100 44s 435ms/step -
 acc: 0.8150 - loss: 0.4027 - val_acc: 0.7110 - val_loss: 0.5891
 Epoch 24/30
 100/100 79s 406ms/step -
 acc: 0.8110 - loss: 0.3961 - val_acc: 0.7300 - val_loss: 0.5331
 Epoch 25/30
 100/100 43s 431ms/step -
 acc: 0.8355 - loss: 0.3776 - val_acc: 0.7300 - val_loss: 0.5430
 Epoch 26/30
 100/100 49s 489ms/step -
 acc: 0.8255 - loss: 0.3809 - val_acc: 0.7150 - val_loss: 0.5526
 Epoch 27/30
 100/100 74s 407ms/step -
 acc: 0.8435 - loss: 0.3583 - val_acc: 0.7460 - val_loss: 0.5352
 Epoch 28/30
 100/100 44s 437ms/step -

```

acc: 0.8475 - loss: 0.3465 - val_acc: 0.7390 - val_loss: 0.5612
Epoch 29/30
100/100          43s 432ms/step -
acc: 0.8565 - loss: 0.3401 - val_acc: 0.7360 - val_loss: 0.5426
Epoch 30/30
100/100          82s 431ms/step -
acc: 0.8660 - loss: 0.3316 - val_acc: 0.7460 - val_loss: 0.5501

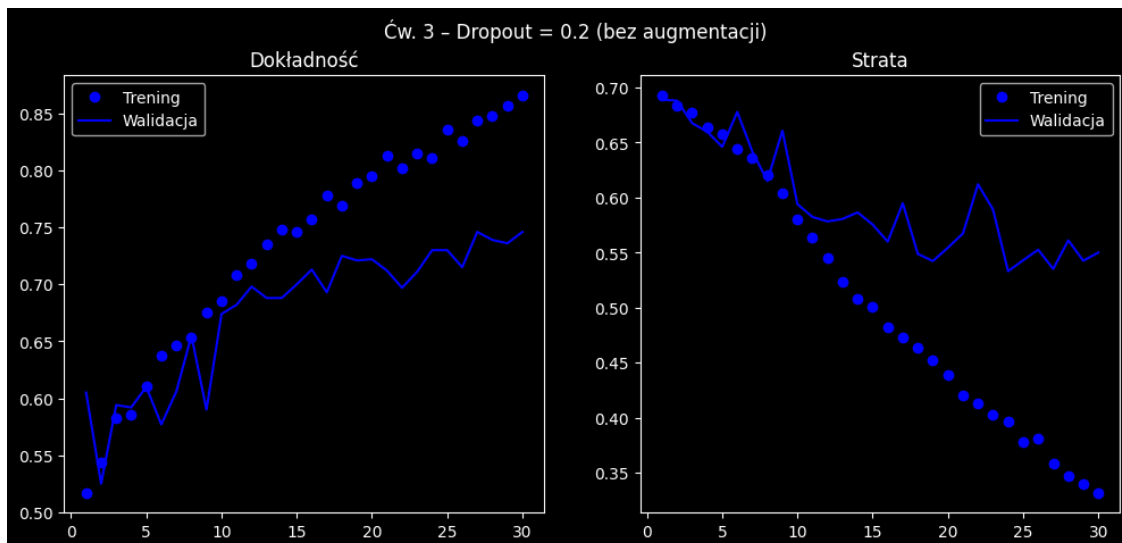
```

```

[12]: acc = history_02.history['acc']
val_acc = history_02.history['val_acc']
loss = history_02.history['loss']
val_loss = history_02.history['val_loss']
epochs = range(1, len(acc) + 1)

plt.figure(figsize=(12, 5))
plt.suptitle('Ćw. 3 - Dropout = 0.2 (bez augmentacji)')
plt.subplot(1, 2, 1)
plt.plot(epochs, acc, 'bo', label='Trening')
plt.plot(epochs, val_acc, 'b', label='Walidacja')
plt.title('Dokładność')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(epochs, loss, 'bo', label='Trening')
plt.plot(epochs, val_loss, 'b', label='Walidacja')
plt.title('Strata')
plt.legend()
plt.show()

```



Trening z Dropout = 0.5 (bez augmentacji)

```
[13]: model_05 = create_model(0.5)
model_05.compile(loss='binary_crossentropy',
                  optimizer=optimizers.RMSprop(learning_rate=1e-4),
                  metrics=['acc'])

history_05 = model_05.fit(
    train_generator_noaug,
    steps_per_epoch=100,
    epochs=30,
    validation_data=validation_generator,
    validation_steps=50
)
```

```
Epoch 1/30
100/100          45s 440ms/step -
acc: 0.5075 - loss: 0.6969 - val_acc: 0.5620 - val_loss: 0.6900
Epoch 2/30
100/100          43s 431ms/step -
acc: 0.5215 - loss: 0.6899 - val_acc: 0.5730 - val_loss: 0.6863
Epoch 3/30
100/100          85s 460ms/step -
acc: 0.5555 - loss: 0.6820 - val_acc: 0.5740 - val_loss: 0.6775
Epoch 4/30
100/100          43s 423ms/step -
acc: 0.5640 - loss: 0.6752 - val_acc: 0.5750 - val_loss: 0.6792
Epoch 5/30
100/100          86s 464ms/step -
acc: 0.5900 - loss: 0.6684 - val_acc: 0.5970 - val_loss: 0.6619
Epoch 6/30
100/100          79s 435ms/step -
acc: 0.6085 - loss: 0.6553 - val_acc: 0.5910 - val_loss: 0.6531
Epoch 7/30
100/100          51s 506ms/step -
acc: 0.6160 - loss: 0.6510 - val_acc: 0.6290 - val_loss: 0.6474
Epoch 8/30
100/100          45s 448ms/step -
acc: 0.6140 - loss: 0.6474 - val_acc: 0.6370 - val_loss: 0.6361
Epoch 9/30
100/100          52s 516ms/step -
acc: 0.6600 - loss: 0.6286 - val_acc: 0.6100 - val_loss: 0.6453
Epoch 10/30
100/100          51s 508ms/step -
acc: 0.6425 - loss: 0.6242 - val_acc: 0.6680 - val_loss: 0.6165
Epoch 11/30
100/100          52s 516ms/step -
acc: 0.6715 - loss: 0.6041 - val_acc: 0.6680 - val_loss: 0.6031
Epoch 12/30
100/100          46s 461ms/step -
```

acc: 0.6970 - loss: 0.5862 - val_acc: 0.6800 - val_loss: 0.5929
 Epoch 13/30
 100/100 47s 464ms/step -
 acc: 0.6925 - loss: 0.5778 - val_acc: 0.6640 - val_loss: 0.5921
 Epoch 14/30
 100/100 53s 530ms/step -
 acc: 0.7100 - loss: 0.5685 - val_acc: 0.6870 - val_loss: 0.5812
 Epoch 15/30
 100/100 44s 437ms/step -
 acc: 0.7080 - loss: 0.5567 - val_acc: 0.6890 - val_loss: 0.5791
 Epoch 16/30
 100/100 44s 438ms/step -
 acc: 0.7160 - loss: 0.5502 - val_acc: 0.6940 - val_loss: 0.5718
 Epoch 17/30
 100/100 44s 442ms/step -
 acc: 0.7270 - loss: 0.5389 - val_acc: 0.6930 - val_loss: 0.5703
 Epoch 18/30
 100/100 44s 438ms/step -
 acc: 0.7325 - loss: 0.5333 - val_acc: 0.6900 - val_loss: 0.5705
 Epoch 19/30
 100/100 44s 442ms/step -
 acc: 0.7405 - loss: 0.5200 - val_acc: 0.7070 - val_loss: 0.5579
 Epoch 20/30
 100/100 44s 441ms/step -
 acc: 0.7415 - loss: 0.5187 - val_acc: 0.6260 - val_loss: 0.6377
 Epoch 21/30
 100/100 44s 438ms/step -
 acc: 0.7465 - loss: 0.5128 - val_acc: 0.7090 - val_loss: 0.5527
 Epoch 22/30
 100/100 50s 495ms/step -
 acc: 0.7560 - loss: 0.5090 - val_acc: 0.7060 - val_loss: 0.5508
 Epoch 23/30
 100/100 44s 440ms/step -
 acc: 0.7515 - loss: 0.5040 - val_acc: 0.7100 - val_loss: 0.5494
 Epoch 24/30
 100/100 44s 440ms/step -
 acc: 0.7560 - loss: 0.4977 - val_acc: 0.7190 - val_loss: 0.5362
 Epoch 25/30
 100/100 45s 444ms/step -
 acc: 0.7600 - loss: 0.4853 - val_acc: 0.7190 - val_loss: 0.5343
 Epoch 26/30
 100/100 44s 438ms/step -
 acc: 0.7750 - loss: 0.4765 - val_acc: 0.6900 - val_loss: 0.5749
 Epoch 27/30
 100/100 49s 491ms/step -
 acc: 0.7690 - loss: 0.4769 - val_acc: 0.7240 - val_loss: 0.5287
 Epoch 28/30
 100/100 74s 410ms/step -

```

acc: 0.7665 - loss: 0.4813 - val_acc: 0.7330 - val_loss: 0.5295
Epoch 29/30
100/100          49s 491ms/step -
acc: 0.7910 - loss: 0.4563 - val_acc: 0.7360 - val_loss: 0.5183
Epoch 30/30
100/100          44s 433ms/step -
acc: 0.7870 - loss: 0.4527 - val_acc: 0.7420 - val_loss: 0.5173

```

```

[14]: acc = history_05.history['acc']
      val_acc = history_05.history['val_acc']
      loss = history_05.history['loss']
      val_loss = history_05.history['val_loss']
      epochs = range(1, len(acc) + 1)

      plt.figure(figsize=(12, 5))
      plt.suptitle('Ćw. 3 - Dropout = 0.5 (bez augmentacji)')
      plt.subplot(1, 2, 1)
      plt.plot(epochs, acc, 'bo', label='Trening')
      plt.plot(epochs, val_acc, 'b', label='Walidacja')
      plt.title('Dokładność')
      plt.legend()
      plt.subplot(1, 2, 2)
      plt.plot(epochs, loss, 'bo', label='Trening')
      plt.plot(epochs, val_loss, 'b', label='Walidacja')
      plt.title('Strata')
      plt.legend()
      plt.show()

```

